



UNIVERSITAT POLITÈCNICA DE CATALUNYA
BARCELONATECH
Centre de la Imatge i la Tecnologia Multimèdia

Lab Session 4: Practical NAT concepts



Multiplayer Games

- Same machine
- Local Network
- 1 Server + N Clients
- P2P

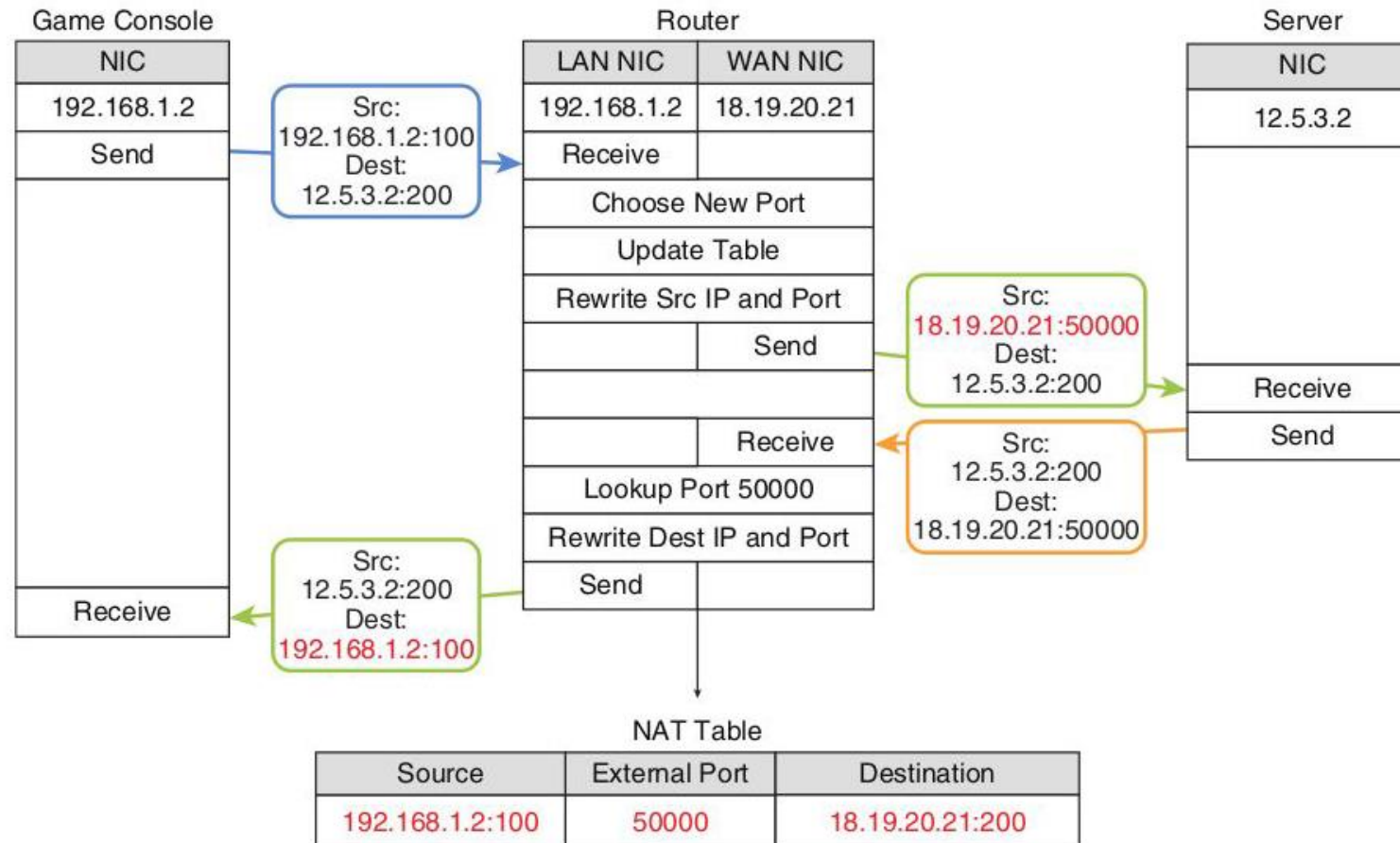
Multiplayer Games

- Same machine → Limited number of Players
- Local Network → Need of physical proximity
- 1 Server + N Clients → Possible Overload on Server
- P2P → NAT (And peer machine dependence)

Multiplayer Games

- ~~Same machine~~ → ~~Limited number of Players~~
- ~~Local Network~~ → ~~Need of physical proximity~~
- 1 Server + N Clients → Possible Overload on Server
- P2P → NAT (And peer machine dependence)

NAT



Methods

- Static/NAT 1:1 – The private IP always gets translated into the same public address. Typical usage is a web server
- Dinamic NAT – The private IP is mapped to an IP based on a registry of public IPs, selecting one that is free
- Overload NAT – All the private IPs will use the same public IP, the router maps the host to a port.

NAT Traversal

- Hole Punching -> Send a message (lost) to an “unknown” client to tell the NAT that it should allow incoming messages
- STUN servers -> Servers dedicated exclusively to share your public IP and routing port to known clients
- NAT types -> (I, II, III, full cone, symmetric, etc.) There are multiple types of NAT, and Hole punching will not work in all of them
- Port prediction -> Some times, ports used can be predicted in the most restrictive cases
- TURN servers -> relay information from clients (more load than STUN, less load than dedicated server)

Importance of NAT

- Pros:
 - It gives you a unique identity in the internet
 - It protects you from unwelcome connections
 - Reduces the use of IP directions
- Cons:
 - It forbids connections from clients unknown to the NAT
 - No clear standard. Different clients have different NAT implementations that require different “tricks” to bypass them
 - Bigger computational demands on routers

Deliverable 3: Serialization

Minimum Requirements

- Launch the scene of your videogame (from a lobby)
- Take a character or object from the client that can be controlled and send information to the server about either:
 - The Position or some other property of the Game Object that is noticeable
 - The Command (key press, click) used by the client to control the Game Object (in this case, the object in the server must move according to the commands on the client)
 - (Both options are valid, with different pros and cons. Use the one that suits better your game. In either case, you will have to serialize and deserialize some data)
- On the server, control the client's object using the information sent by the client
- On the server, control a different object than the client and send it to the client
- On the client, receive the data from the second object and change it as the server commands
- Do all this using UDP

Extras

- Complete moveset available
- Send other Actions over the network (shoot, brake, interactions...)
- Speculate on how to keep lag low
- Speculate on how to ensure messages are received
- Have a complete game experience (even if there is lag)
- Any other proposition of yours (ask me first)

Evaluation of Delivery 3

- 40% Serialize and send some data from client to server that is not text. Show the change this data has on the server
- 25% Do the same for the server. Have a 2 simultaneous player experience.
- 25% Extras
- 10% Code is clean and understandable

Demo Day

- For the demo:
 - Try to have a complete game experience, even if it is only for a tiny part of your game
 - Have at least 2 players (one as a server, and the other as a client)
 - Try to make the game experience fun to start for one-two minutes

Reminder of Deliverables

1. Threads
2. Sockets (5%)
3. **Serialization (15%) - Mid-term Demo**
 1. 5% delivery, 5% expert grading, 5% popular vote
4. World State Replication (10%)
5. **Latency/jitter mitigation (20%) - Final Demo**
 1. 10% delivery, 5% expert grading, 5% popular vote